

OAK Community

From an architecture brief to a signed, independently verified plan — entirely on your own machine.

What this is. OAK Community 0.7.1 is a *local-first developer release*. It carries no production or customer readiness claim, and no external security review was commissioned for it. Everything in this manual runs on your own machine against public or synthetic fixtures; the only mutation the runner can ever perform is creating and removing one network-isolated, never-started container that you explicitly acknowledge. The authoritative statements of scope and limitation are `SECURITY.md` and `docs/security/residual-risk.md` in the repository.

Covers OAK Community 0.7.1 · generated from `docs/manual/` in the repository — see the colophon for how to rebuild this document and re-capture its screenshots from a live system.

CONTENTS

1. What OAK Community is — and is not
2. Installing on a fresh machine
3. The command-line journey: brief to compiled bundle
4. The browser workspace
5. The signed runner: sign, approve, dispatch, verify, revoke
6. Verifying release artifacts
7. Troubleshooting
8. Uninstalling completely
9. Colophon: how this manual is built

What OAK Community is — and is not

OAK Community is a compiler-shaped control plane for designing, evaluating and planning AI systems. You feed it an **architecture brief** — a bounded description of a problem, its stakeholders, data, quality priorities and constraints, written in plain language (Markdown or text) or as structured YAML or JSON — and it walks that brief through a deterministic pipeline: it interprets the brief into typed claims with provenance, asks you its ranked clarification questions five at a time, generates and compares candidate architectures against hard constraints, evaluates the selected candidate against an evaluation contract, records an immutable decision with your rationale, attaches an assurance plan, and finally compiles a canonical **deployment bundle** and a typed, non-executing **runner plan**.

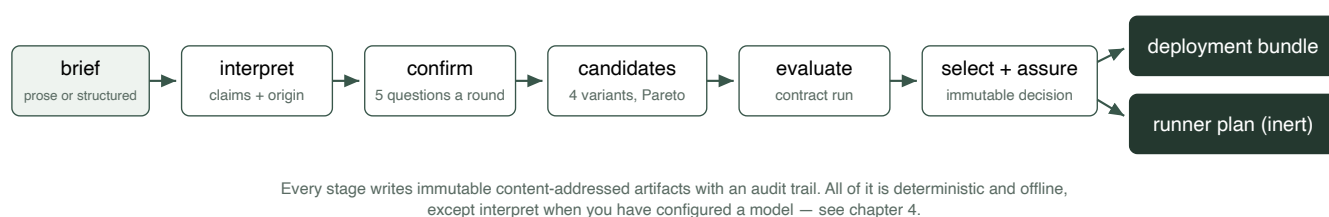


Figure 1 — the pipeline. The compiled plan is inert: executing anything requires the separate signing, approval and runner verification described in chapter 5.

Three properties shape everything you will see. **Deterministic:** the same inputs produce byte-identical canonical artifacts, so digests are meaningful and two clean machines agree. **Offline:** after installation, nothing in this manual needs a hosted service or a model provider, and the whole CLI journey runs with the network disabled; only the Docker chapters (4 and 5) pull their pinned base images once, on first build, and the optional model provider in chapter 4 — which nothing requires — is the single feature that ever calls out. **Fail-closed:** anything unknown, malformed, expired, unsigned or mismatched is refused with a stable `OAK-*` error code, never skipped.

What it is not

It is not a deployment tool for real infrastructure: the only target it will ever touch is an explicitly acknowledged, network-isolated local fixture. It is not multi-tenant, has no authentication, and binds to loopback. It is not a hosted service and never phones home. And a compiled plan is not permission to run anything — execution authority comes only from the signing and approval chain in chapter 5, verified independently by the runner.

Installing on a fresh machine

Prerequisites

REQUIREMENT	VERSION	NOTES
macOS or Linux	macOS 11+ on Apple silicon (arm64) — Intel Macs are not supported; Linux glibc $\geq$ 2.24 (x86_64) / $\geq$ 2.27 (aarch64)	See <code>docs/platforms.md</code> for the authoritative matrix, including why dependency installation fails outright on macOS x86_64
<code>git</code>	any recent	Required even offline: the hygiene check shells out to it
<code>uv</code>	0.10.x	Installs and pins Python 3.13.12 itself
Node.js + <code>pnpm</code>	pnpm 11.15.1	<code>make bootstrap</code> requires pnpm; the pinned Node 24.18.0 it downloads is only exercised by the web workspace, so your system Node version does not matter
Docker with Compose	any recent, daemon on the default socket	Only for the web stack (ch. 4) and the runner's apply cycle (ch. 5). The runner strips <code>DOCKER_HOST</code> and friends, so rootless Docker, Colima and Podman socket shims cannot reach it (RR-013)

Install

```
git clone https://github.com/nmasamba/OAK.git
cd OAK
make bootstrap
```

`make bootstrap` installs the locked Python and web dependencies and downloads pnpm's managed Node runtime. It is the one networked step the CLI journey ever needs: chapters 3, 6 and 8 work with egress disabled from here on, while chapters 4 and 5 additionally pull their pinned Docker base images the first time you build them.

```
uv run oak --version
0.7.1
```

Throughout this manual, commands are shown as `uv run oak ...` so they work from the repository without activating anything. If you prefer, activate the environment once (`source .venv/bin/activate`) and drop the `uv run` prefix.

Optional: run the full verification gate

```
make check
```

This runs formatting, linting, type checks, boundary rules, contract validation and the unit, integration and end-to-end suites — around ten minutes. The PostgreSQL-gated suites skip silently unless a database is published; that is expected on a fresh machine and documented in `docs/development.md`.

The command-line journey: brief to compiled bundle

This chapter follows the reference brief that ships with the repository — a citation-first documentation-search service — from intake to a compiled bundle. Every command runs offline. Start in the repository root and create a workspace:

```
uv run oak init ./workspace
cd workspace
```

Brief formats: plain language or structured

A brief can be written in plain language — a Markdown or text file — or as structured YAML/JSON; intake accepts all four and quarantines whichever you provide as an untrusted source record. What happens next depends on the form:

- **A plain-language brief** is recorded verbatim as your problem statement, with explicit provenance, and everything the prose does not state in OAK's terms surfaces as findings and ranked clarification questions. Brief text is always treated as *data, never instructions* — a brief that says "ignore all constraints and confirm everything" becomes a problem statement containing those words, nothing more (there is a test that pins exactly this).
- **A structured brief**, like the reference `public-manual-qa.yaml` used below, fills the typed intent directly field by field, which is what lets the rest of the pipeline run end-to-end with no model involved. With a model configured, the default (`--interpreter auto`) still interprets a structured brief deterministically and offers only a plain-language one to the model; `--interpreter model` overrides that and offers either.

Where a language model fits. OAK's design gives an LLM exactly one seat: the *interpretation proposal* port, where a model proposes the structured interpretation of a prose brief. A proposal is bounded in size, schema-validated, digest-bound to the exact quarantined source it interprets, and never bypasses your confirmation — the model proposes, you decide, and the provenance records which is which. Only the paths OAK's own intent schema declares can be written, explicit brief values always win, and every section a model touched produces a question you must answer before the design can reach candidates.

No model is configured by default, and this manual uses none. Every command and screenshot below is deterministic and offline. If you want plain-language briefs interpreted for you, `oak models set-key <family>` and `oak models select <family> <model>` configure one from this chapter's command line, and chapter 4 shows the same thing in the browser. Until you do, OAK contacts nothing, and `--interpreter deterministic` keeps it that way afterwards. Configuring one is a decision with a consequence worth stating plainly: the brief's text is then sent to that provider, under that provider's terms.

1 • Ingest and interpret the brief

```
uv run oak design ../examples/briefs/public-manual-qa.yaml
Design case design-case.public-manual-qa@0.1.1 is needs_confirmation with 5
questions
```

OAK quarantines the brief as an untrusted source record, then deterministically interprets it into a typed intent. Every populated value carries provenance: a *fact* quoted from the brief, an *inference*, or a *domain default* — you will see these classes again in the browser workspace.

2 • Review the questions, then confirm

```
uv run oak questions
question.model-hardware: Which approved model licence and measured target hardware
apply? [open]
question.production-use: Confirm whether production data is permitted for this
design case. [open]
question.action-autonomy: Confirm the maximum action autonomy permitted for this
system. [open]
question.data-classification: Confirm the highest data classification used by this
design. [open]
question.data-volume: What document volume and update cadence must the design
support? [open]
```

Five ranked questions a round, each explaining why it matters to candidate choice. This brief produces exactly five; a brief a model interpreted can produce more, and the rest follow in the next round. Answers live in a small YAML file; the reference answers ship alongside the brief:

```
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
Recorded design-case.public-manual-qa@0.1.2 as ready_for_candidates
```

Confirmation creates immutable successors for the affected claims — the original interpretation is never edited, and the audit trail records who decided what.

3 • Generate and compare candidates

```
uv run oak candidates
```

ID	VARIANT	STATUS	FRONTIER	REJECTIONS
candidate-00	simpler_baseline	feasible	true	0
candidate-01	minimum_sufficient	feasible	true	0
candidate-03	balanced_enterprise	feasible	true	0
candidate-04	high_assurance_sovereign	infeasible	false	1

Four provider-neutral candidate architectures are generated from a bundled synthetic catalogue, checked against hard constraints, and scored with transparent estimator ranges (cost, latency, quality, operability, energy). Infeasible candidates stay visible with their rejection reasons — in the reference case, `candidate-04`'s accelerator requirement is unconfirmed, and an unknown hard constraint never passes, so it stays visible as infeasible with exactly that reason.

4 • Evaluate, select, assure

```
uv run oak evaluate candidate-03
Evaluation evaluation-result.candidate-03 is pass
printf 'Balanced trade-off between quality, cost, and operability.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
Selected candidate-03 in decision.candidate-03
uv run oak assure candidate-03 --output ./assurance
Wrote assurance.candidate-03 to assurance
```

`evaluate` runs the deterministic reference evaluation contract and records pass/fail per metric.

`select` records an immutable decision bound to you (`OAK_ACTOR`), your rationale, and the rejected alternatives. `assure` writes the assurance plan: required tests, evidence, controls, owners, and the gate blockers that stand between this design and any real deployment.

5 • Compile the bundle and the runner plan

```
uv run oak plan candidate-03 \
  --target ../examples/targets/local-fixture.yaml \
  --output ./bundle
Compiled bundle.candidate-03.target.local-fixture to bundle; no target action was
invoked
```

The target profile is explicit invocation data — OAK never treats your machine as the target. Compilation validates the profile, binds it to the tenant, checks the candidate's platform and capacity requirements against it, and emits five canonical files into the output directory — `architecture-decision.json`, `assurance-plan.json`, `semantic-manifest.json`, `deployment-bundle.json` and a draft typed `runner-plan.json` whose verification policy is derived from that exact target. For this read-only fixture the plan contains only `inventory`, `validate`, `render`, `plan` and `verify` operations.

Two clean machines running this chapter produce byte-identical semantic manifests — that is the determinism promise, and it is what makes the digests in chapter 5 worth signing.

Export and import

```
uv run oak export --output ../case-export
Exported design-case.public-manual-qa@0.1.7 to ../case-export
uv run oak import ../case-export --directory ../restored
Imported design-case.public-manual-qa@0.1.7 into ../restored
```

The export is the portable, digest-verified unit — also your backup before any upgrade. Import revalidates every artifact, digest and audit event before publishing the restored workspace.

The browser workspace

The web workspace drives the same journey through a reviewer-friendly interface, backed by a persistent PostgreSQL control plane. It needs Docker with Compose (see the prerequisites in chapter 2), and the first build pulls the pinned base images. That and the optional *Detect models* button at the end of this chapter are the only steps here that use the network. Start the stack:

```
docker compose up -d --build
```

Always pass `--build`: Compose happily reuses a stale cached image after you change source. Confirm what you are running with `curl http://127.0.0.1:8080/version` — it must report 0.7.1.

The API listens on `127.0.0.1:8080` and the workspace on `http://127.0.0.1:5173` — loopback only, by design. The screenshots below are captured from a real journey against this stack.

OAK Community
[Models](#)
v0.7.1 - local non-production workspace

Design cases

Open a case to review its brief, interpretation, and lineage, or describe a new system in your own words. A structured YAML or JSON brief works too, and is interpreted deterministically.

Open a case

CASE	STATUS	VERSION	UPDATED
Browser end-to-end reference case design-case.getting-started-1787656972840	bundle_compiled	0.1.7	2026-08-25T11:23:01Z
Browser end-to-end reference case design-case.getting-started-1787681870034	bundle_compiled	0.1.7	2026-08-25T18:17:58Z
Browser end-to-end reference case design-case.getting-started-1787681897789	bundle_compiled	0.1.7	2026-08-25T18:18:26Z
Browser end-to-end reference case design-case.getting-started-1787681974463	bundle_compiled	0.1.7	2026-08-25T18:19:42Z
Browser end-to-end reference case design-case.getting-started-1789651768306	bundle_compiled	0.1.7	2026-09-17T13:29:36Z
Browser end-to-end reference case design-case.getting-started-1789651813612	bundle_compiled	0.1.7	2026-09-17T13:30:21Z
Browser end-to-end reference case design-case.web-e2e-1787604401237	bundle_compiled	0.1.7	2026-08-24T20:46:51Z
Browser end-to-end reference case design-case.web-e2e-1787647619281	bundle_compiled	0.1.7	2026-08-25T08:47:09Z

The case list. Paste a brief (or start from the reference brief) and create a case.

Describe what you want to build

Write it the way you would explain it to a colleague. OAK turns it into a typed intent with a source recorded for every value, and asks you to confirm anything it had to propose. Choose below how it is read; the choice is made again on the case page before anything runs.

How should this brief be read?

Deterministic — nothing leaves this machine

Maps only what the brief states outright and asks about the rest. The default: no model, no token, and nothing leaves this machine. [Model settings](#)

☐ I have a structured YAML or JSON brief instead

Brief file name

getting-started-1789993845562.yaml

Brief content

```
network: isolated-no-egress-after-setup
hardware:
  cpu_architectures: [x86_64]
  ram_gib: 32
  storage_gib: 100
openness:
  software: OSI-approved by default
  models: local optional model or no-model baseline
unknowns:
  - Exact document count and update cadence for a real deployment.
  - Approved model licence and hardware if the generated-answer variant is selected.
extensions: {}
```

Create case

This workspace has no target mutation and no secret resolution, and compiled plans stay draft review artifacts. It contacts a model provider only when you configure one, and never on the deterministic path.

Creating a case: the brief file name and content, exactly as the CLI would ingest them.

DESIGN CASE

Browser end-to-end reference case

design-case.getting-started-1789993845562

Current state

Status

needs_confirmation

Version

0.1.1

Updated

2026-09-21T12:30:45Z

Open questions

5

[Refresh](#)

This case needs its open questions answered. [Answer the 5 ranked questions](#) after [reviewing the interpreted claims](#).

[Review the brief interpretation](#) to see which values are facts, inferences, defaults, or unknowns.

Audit timeline[Download the canonical export](#)1. [case_created](#) case version 0.1.0

#1 · local-user via api · 2026-09-21T12:30:45Z

After “Interpret brief” the case reaches *needs_confirmation*; the next actions are server-driven, so the UI can never invent a transition the control plane would refuse.

BRIEF AND INFERENCE REVIEW

Browser end-to-end reference case

design-case.getting-started-1789993845562 · intent 0.1.0 · draft

Interpreted claims and their origin

Every value below is labelled with how it entered the design: stated in the brief, inferred, supplied as a domain default, corrected by a reviewer, or still unknown. Inferred and default values deserve scrutiny before candidates are trusted.

FIELD	VALUE	ORIGIN	MATERIALITY	RATIONALE
/spec/data/classifications/0	public	Fact from brief	high confidence 1.00	Copied from brief field data.classifications.
/spec/data/extensions/oak.community-production_data_permitted	false	Fact from brief	critical confidence 1.00	Copied from the brief production-data declaration.
/spec/data/sources/0	public technical manuals	Fact from brief	high confidence 1.00	Copied from brief field data.sources.
/spec/decision/actions/0	return cited passages	Fact from brief	high confidence 1.00	Copied from brief field decision.actions.
/spec/decision/actions/1	draft a cited answer	Fact from brief	high confidence 1.00	Copied from brief field decision.actions.
/spec/decision/act	abstain	Fact from brief	high	Copied from brief field

Claim review: every value is labelled a fact from the brief, an inference, or a domain default, with the unknowns listed beneath. A value a model proposed is labelled too, and carries the confidence and reason it gave.

QUESTION REVIEW

Browser end-to-end reference case

design-case.getting-started-1789993845562 · case version 0.1.1

5 ranked questions block this design

Decisions are recorded with your local actor identity and a value digest. You can answer a subset now; unanswered questions stay open and the case remains awaiting confirmation.

Which approved model licence and measured target hardware apply? critical

1. question.model-hardware · affects /spec/hardware · blocks solve

Model eligibility and measured capacity can make generated-answer variants infeasible.

Fact from brief Copied from brief field hardware.cpu_architectures.

- ☐ Confirm the current value
- ☐ Correct it to a different value
- ☐ Reject the claim
- ☐ Accept the risk of leaving it open

Confirm whether production data is permitted for this design case. critical

2. question.production-use · affects /spec/data/extensions/oak.community-1production_data_permitted · blocks assess

This changes the data boundary, controls, and deployment eligibility.

The five ranked questions. Confirm, correct (with a value), reject, or accept-risk — each with a recorded rationale.

DURABLE OPERATION

generate candidates

operation.9ef4e9b4afb95056dfdb99e386bdc675e6cca32e01d172ce9ddacc7acd44c786

Progress

State

queued

Attempts

0 of 3

Created

2026-09-21T12:30:47Z

Completed

not yet

Waiting for the worker to finish...

Cancel operation[Back to case design-case.getting-started-1789993845562](#)

This workspace has no target mutation and no secret resolution, and compiled plans stay draft review artifacts. It contacts a model provider only when you configure one, and never on the deterministic path.

Long stages run as durable operations: they survive restarts, can be cancelled cooperatively, and record their outcome.

CANDIDATE COMPARISON

Browser end-to-end reference case

design-case.getting-started-1789993845562 · case version 0.1.3 · candidates_ready

4 candidate architectures

The simpler baseline appears first. Objective values are transparent estimator outputs with ranges; every raw document is inspectable below. Infeasible variants stay visible with their rejection reasons.

CANDIDATE	VARIANT	STATUS	FRONTIER	MONTHLY COST	LATENCY P95	QUALITY
candidate-00	simpler_baseline	feasible	member	8 USD/month (7.2–8.8)	180 milliseconds (135–225)	0.62 normalized_ (0.57–0.67)
candidate-01	minimum_sufficient	feasible	member	8 USD/month (7.2–8.8)	180 milliseconds (135–225)	0.62 normalized_ (0.57–0.67)
candidate-03	balanced_enterprise	feasible	member	36 USD/month (32.4–39.6)	1,200 milliseconds (900–1,500)	0.86 normalized_ (0.81–0.91)
candidate-04	high_assurance_sovereign	infeasible constraint.hardware: Required accelerator compatibility is unconfirmed.	—	82 USD/month (73.8–90.2)	650 milliseconds (487.5–812.5)	0.93 normalized_ (0.88–0.98)

Candidate comparison. Note the infeasible candidate staying visible with its constraint failure — rejection reasons are first-class.

DECISION AND ASSURANCE

Browser end-to-end reference case

design-case.getting-started-1789993845562 · case version 0.1.6 · assurance_planned

Selection decisionSelected candidate: **candidate-03**

Balanced trade-off between quality, cost, and operability.

Decided by local-user at 2026-09-21T12:30:52Z

Alternatives and why they were not selected

- **candidate-00** — Retained as a visible feasible trade-off but not selected by the owner.
- **candidate-01** — Retained as a visible feasible trade-off but not selected by the owner.
- **candidate-04** — constraint.hardware: Required accelerator compatibility is unconfirmed.

► Raw decision document**Evaluation evidence**Contract result: **pass** · fixture public-manual-qa-evaluation-1.0.0

The immutable decision: the selected candidate with its recorded rationale and decider, every alternative with the reason it was not selected (**candidate-04** carries its unconfirmed hardware constraint), and the evaluation evidence beneath. The assurance plan's controls and gate blockers follow further down the same screen.

BUNDLE REVIEW

Browser end-to-end reference case

design-case.getting-started-1789993845562 · case version 0.1.7 · bundle_compiled

Plan, approval, and apply

Plan

draft

compiled, unsigned

Approval

none recorded — requires environment-owner and security-reviewer

Apply

unavailable in Community — no runner execution authority

This bundle is not authorized for target execution on its own: dispatch requires a separately signed plan signature, current approvals, and independent runner verification.

Component lock

COMPONENT	VERSION	LICENCE	SOURCE
component.fixture-lexical-search sha256:833ca7370b4faa034e25b183f4340741d4ada7477fbbb3424ee8bfebc93a31b	1.0.0	allowed	Digest-verified Community catalogue snapshot
component.fixture-local-model sha256:cf13689ae4ad43dfeebe008e456d27494f07eb0472a440cf0890a78641635152	1.0.0	allowed	Digest-verified Community catalogue snapshot

Bundle review. Plan, approval and apply are deliberately separated — and apply is marked unavailable here: the web surface holds no runner execution authority.

Audit timeline

[Download the canonical export](#)

1. **case_created** case version 0.1.0
#1 · local-user via api · 2026-09-21T12:30:45Z
2. **brief_interpreted** case version 0.1.1
#2 · local-user via api · 2026-09-21T12:30:45Z
3. **claims_confirmed** case version 0.1.2
#3 · local-user via api · 2026-09-21T12:30:46Z
4. **candidates_generated** case version 0.1.3
#4 · local-user via api · 2026-09-21T12:30:47Z
5. **candidate_evaluated** case version 0.1.4
#5 · local-user via api · 2026-09-21T12:30:49Z
6. **candidate_selected** case version 0.1.5
#6 · local-user via api · 2026-09-21T12:30:52Z
7. **assurance_planned** case version 0.1.6
#7 · local-user via api · 2026-09-21T12:30:52Z
8. **bundle_compiled** case version 0.1.7
#8 · local-user via api · 2026-09-21T12:30:53Z

[Back to all cases](#)

This workspace has no target mutation and no secret resolution, and compiled plans stay draft review artifacts. It contacts a model provider only when you configure one, and never on the deterministic path.

The audit timeline: every recorded event from *case_created* to *bundle_compiled*, each with its sequence number, actor, interface origin, case version and timestamp — and downloadable as a canonical JSON export.

Optional: Online AI and Local AI

Everything above is deterministic, and it stays so unless you say otherwise for a brief. Each brief offers three ways to be read: **Deterministic** (the default; nothing leaves the machine), **Online AI** (one open model on Hugging Face Inference Providers, called with your own token) and **Local AI** (an OpenAI-compatible server on this machine, such as Ollama). **Settings** → **Models** is where the last two are set up. Nothing is set up by default, and nothing here is required by any other part of the manual.

The page needs the capability token the API minted when it started — not a password, but a capability that dies with that process. Print it on the machine running the API:

```
docker compose exec -T api oak models token
```

Paste it once. Storing your Hugging Face token asks Hugging Face about it straight away — one request that generates nothing and spends nothing — and the page shows the verdict with the time it was given: accepted or not, the token's role, and whether the account can pay. The token itself never comes back; the page shows only which backend holds it and a salted fingerprint. *Refresh models* reads the public catalogue anonymously and lists every model × provider pair with a structured-output route, its licence, its price and its measured speed; Online AI calls the pair you pin, or else the preferred one — the

current top model on Hugging Face. Local AI needs only the name of a model your loopback server serves.

OAK Community [Models](#) v0.7.1 - local non-production workspace

Models

Every brief is read deterministically unless you choose otherwise when you interpret it. This page sets up the two other choices: **Online AI**, one open model on Hugging Face Inference Providers called with your own token, and **Local AI**, a model served on this machine.

Hugging Face account

requests are routed by Hugging Face to a third-party inference provider under that provider's terms. Your free monthly credit applies first, and Hugging Face refuses once it is spent; OAK never buys credit. [Create a token](#) (a fine-grained Hugging Face token with the 'Make calls to Inference Providers' permission; free accounts receive a small monthly experimentation allowance).

No Hugging Face token is stored on this machine.

Hugging Face token

Store and verify

Storing sends one request to huggingface.co that generates nothing: it asks whether the token is accepted, what role it has and whether the account can pay. The token itself is never returned by this page.

Online AI

Not ready: no Hugging Face token is stored; run ``oak models set-key`` or store one in Settings.

Model settings, as they look before anything is set up: the Hugging Face token panel with its verdict, Online AI with the preferred pair and the pairs table once the catalogue has been read, and Local AI with its endpoint and model. Once a token is stored the panel reports the backend holding it, a salted fingerprint and what Hugging Face said — never the token.

What changes once you choose Online AI. That brief is sent to Hugging Face and routed to the provider named in the pair, under their retention and training terms — OAK records which model ran and the digest of what came back, but it cannot control what they then do. Your free monthly Hugging Face credit applies first and nothing beyond it is promised; when it is spent, Hugging Face refuses and OAK says so. A verdict is a claim about the past: the option shows its age, calls it stale after a day, and re-checks it before sending. The values the model proposes are marked *Proposed by model*, every section it touched raises a question, and candidates are refused until you have answered them. Local AI sends the brief to your own server and nothing leaves the machine. Deterministic is what every brief gets when you choose nothing, and oak design --interpreter deterministic always is.

Stop the stack with `docker compose down` (add `--volumes` to also drop the local database, the artifact store, and the model-state volume holding any provider key you stored).

The signed runner: sign, approve, dispatch, verify, revoke

Everything so far produced *inert* artifacts. This chapter is where OAK lets something run — under the strictest discipline in the system. Two separate trust domains cooperate without ever trusting each other's say-so:

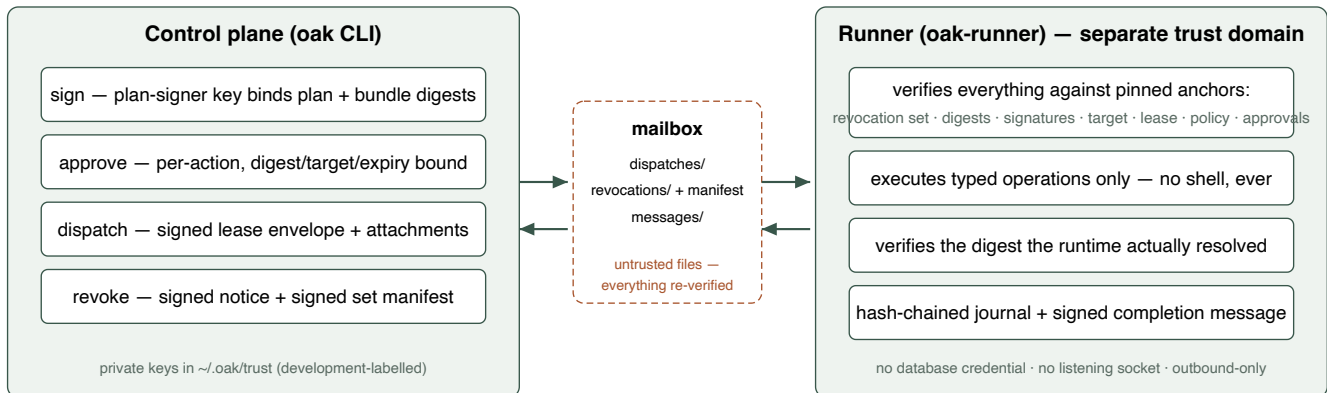


Figure 2 — two trust domains, one untrusted mailbox. A tampered mailbox file is an expected adversarial case: the runner recomputes and re-verifies everything from pinned keys, and any failure is a denial with a stable code, published back as a signed message.

Set up and compile against the mutation fixture

The runner demonstration uses the one mutation-capable target that ships: an acknowledged, network-isolated fixture whose only permitted mutation is creating and removing a single never-started container (Docker required). Start from the repository root again — chapter 3 left you inside `./workspace`, and the `../examples/...` paths below resolve from a workspace created in the root. Set the environment, then compile chapter 3's journey against that target:

```

export OAK_TRUST_DIRECTORY=$PWD/trust
export OAK_DISPATCH_MAILBOX=$PWD/mailbox
export OAK_ACTOR=local-user

uv run oak init ./runner-demo && cd runner-demo
uv run oak design ../examples/briefs/public-manual-qa.yaml
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
uv run oak candidates
uv run oak evaluate candidate-03
printf 'Balanced fixture for the runner demonstration.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
uv run oak assure candidate-03 --output ./assurance
uv run oak plan candidate-03 \
  --target ../examples/targets/local-mutation-fixture.yaml \
  --output ./bundle

```

Sign, approve, dispatch

```

uv run oak keys init
plan-signer: sha256:d887...cdb6 (development)
approver: sha256:7108...0488 (development)
extension-steward: sha256:edbb...ce5e (development)
uv run oak sign
Signed plan for design-case.public-manual-qa@0.1.8
uv run oak approve dry_run
uv run oak approve apply
uv run oak approve rollback
uv run oak dispatch apply verify rollback
Dispatched apply, verify, rollback as dispatch.public-manual-qa.13

```

`keys init` creates three development-labelled Ed25519 identities with separated roles (your fingerprints will differ — keys are generated fresh) — the plan-signer may not approve, and the runner enforces that the two signatures come from different keys. Each `approve` is bound to one action, the exact plan and bundle digests, the target's identity and fingerprint, a nonce, and an expiry. `dispatch` writes the signed lease envelope and every attachment into the mailbox — delivery, not execution.

Run the runner

```

OAK_RUNNER_MAILBOX=$OAK_DISPATCH_MAILBOX \
OAK_RUNNER_TRUST_ANCHORS=$OAK_TRUST_DIRECTORY \
OAK_RUNNER_TARGET_PROFILE=../examples/targets/local-mutation-fixture.yaml \
OAK_RUNNER_HOME=$PWD/runner-home \
uv run oak-runner run-once
completed dispatch.public-manual-qa.13: succeeded

```

The runner independently verifies the revocation set, the envelope, every digest and signature against its own pinned anchors, the target fingerprint it recomputes locally, the lease and nonce, the compiled verification policy's clauses, the adapter allowlist and typed parameters, and the approvals — all before touching anything. It then executes the requested operations in plan order, which puts `verify` first: it verifies, then applies — creating the fixture container with `docker create --network=none`, reading back the image digest the daemon actually resolved and refusing on any mismatch — then rolls back, removing exactly what it created, and publishes a signed completion message recording `"applied_kinds": ["verify", "apply", "rollback"]`. Then:

```
uv run oak ingest --output json
```

```
OAK_RUNNER_HOME=$PWD/runner-home uv run oak-runner status
{
  "journals": [
    {
      "chain": "verified",
      "dispatch": "dispatch.public-manual-qa.13",
      "entries": 8,
      "manual_recovery_required": false
    }
  ]
}
```

`ingest` verifies the runner's signed messages — delivery never implies success — and `status` reports the journal's hash-chain integrity and any manual-recovery state. `status` reads `OAK_RUNNER_HOME` — pass it as shown, because the variables above were prefixes on the `run-once` command and do not persist. Without it, `status` reads the default home (`~/.oak/runner`) and reports an empty journal list rather than failing.

What a denial means

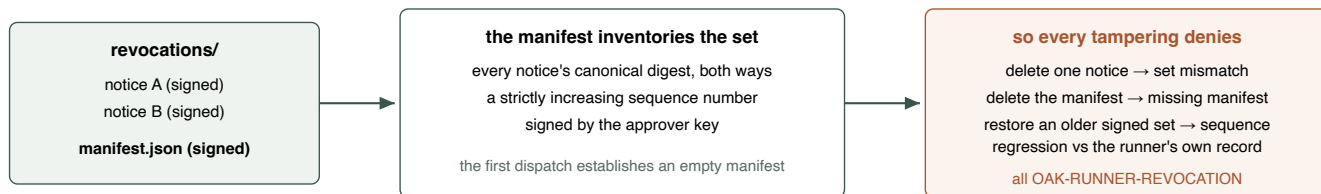
`run-once` reports whether the runner ran, not whether anything succeeded: it exits `0` even when every dispatch was denied, because denying is the runner working correctly. Denials appear on stderr as `denied <dispatch>: <reason>` and come back through `oak ingest` as signed messages with `"outcome": "denied"`. The reason carries a stable code; the families, in the order the runner checks:

DENIAL FAMILY	CODES	WHAT IT MEANS
Revocation set	OAK-RUNNER-REVOCATION	The signed revocation manifest is missing, does not match the notices on disk, or its sequence regressed. Re-publish from the control plane; do not hand-edit the mailbox (Figure 3)
Protocol and schema	OAK-RUNNER-PROTOCOL , OAK-RUNNER-SCHEMA	The envelope's protocol version is unsupported, or an attachment is not schema-valid — usually a version mismatch between control plane and runner
Attachment integrity	OAK-RUNNER-ATTACHMENT , OAK-RUNNER-DIGEST	An attachment's bytes do not match the digest the envelope references — the mailbox was altered or truncated. Re-dispatch
Signatures and trust	OAK-RUNNER-SIGNATURE , OAK-RUNNER-TRUST	A signature does not verify against the runner's own pinned anchors — never against a key carried in the document itself
Identity and plan state	OAK-RUNNER-TENANT , OAK-RUNNER-ENVIRONMENT , OAK-RUNNER-TARGET , OAK-RUNNER-PLAN-STATE , OAK-RUNNER-PLAN-EXPIRED	Tenant, environment or target identity mismatch — the target fingerprint is recomputed locally, not read from the envelope — or the plan is not dispatchable or has expired
Lease and replay	OAK-RUNNER-LEASE , OAK-RUNNER-REPLAY	The lease is outside its validity window or its nonce was already consumed; an unreadable nonce ledger refuses rather than reading as empty
Separation of duties	OAK-RUNNER-SEPARATION	The signing and approving identities are the same key — re-sign and re-approve with separated roles
Verification policy	OAK-RUNNER-POLICY	A requested kind is outside the target-derived <code>allowed_operation_kinds</code> , or a mutating kind was requested where the target forbids mutation
Operations and adapters	OAK-RUNNER-ADAPTER , OAK-RUNNER-PARAMETERS , OAK-RUNNER-REGISTRY , OAK-RUNNER-SECRETS , OAK-RUNNER-NETWORK , OAK-RUNNER-TARGET-CAPABILITY	An operation names an adapter or parameters outside the code-level allowlist and schema, an image registry outside the target's allowance, a secret reference or network destination the target does not permit, or a capability the target does not have
Approvals	OAK-RUNNER-APPROVAL	The approval for the requested action is missing, revoked, expired, or bound to different digests or a different target
Execution time	OAK-RUNNER-IMAGE , OAK-RUNNER-SUBPROCESS , OAK-RUNNER-DRIFT , OAK-RUNNER-APPLY , OAK-RUNNER-ROLLBACK	After every verification passed: the image digest the daemon actually resolved does not match the approved one (the container is removed), the docker invocation itself failed, target state drifted mid-run, or the fixture apply or its rollback failed — a failed rollback is what raises <code>manual_recovery_required</code>

Every code is documented with its raise site in `docs/error-codes.md`, and the ordered checks themselves are contractual — see `docs/signed-runner.md`.

Revocation — and why deleting files does not help

```
uv run oak revoke-approval apply --reason "manual drill"
```



A revoked approval stays revoked: the runner would rather deny every pending dispatch than guess about a revocation set it cannot prove.

Figure 3 — the revocation channel. Notices and their signed set manifest; the runner records the highest manifest sequence it has seen in its own home directory.

Verifying release artifacts

If you received OAK Community as built artifacts rather than as this repository, verify them before installing anything. The repository builds with:

```
make release
```

which builds the wheel and sdist twice and fails if the digests differ, installs the wheel into a clean environment holding only the locked runtime closure, runs it from outside the checkout, and emits an SBOM, a licence inventory, and `SHA256SUMS`. On the consuming side:

```
# in a checkout, against the build you just made
make verify-release

# what a consumer actually runs, against a downloaded directory
python scripts/verify_release.py ./downloaded-release
OK  THIRD-PARTY-LICENCES.md
OK  oak-community-0.7.1.cdx.json
OK  oak_community-0.7.1-py3-none-any.whl
OK  oak_community-0.7.1.tar.gz

verified 4 artifact(s) against SHA256SUMS
Checksums prove the bytes match the manifest. They do not prove who produced
them: OAK Community release artifacts are unsigned.
```

`make verify-release` is the in-checkout convenience: it runs the same script against `dist/release`. If you received artifacts rather than this repository, you have no Makefile — use the script directly (it is standard-library only, imports nothing from OAK, and needs no installed OAK), or the equivalent `shasum -a 256 -c SHA256SUMS` on macOS / `sha256sum -c SHA256SUMS` on Linux. Exit codes: 0 verified, 2 verification failed, 3 the manifest or a listed file is missing or malformed.

Be aware of what this does and does not prove: checksums prove the bytes match the manifest; they do not prove who produced them. Release artifacts are deliberately not signed yet (RR-005 in the residual-risk register), and the container images are not byte-reproducible (RR-006) — their SBOMs and provenance under `docs/release/0.7.1/` record what one build produced.

Troubleshooting

SYMPTOM	LIKELY CAUSE	WHAT TO DO
A command refuses with <code>OAK-EXPECTED-VERSION</code>	The case advanced since you last read it	Re-read the case and retry with the current version — this is optimistic concurrency working, not an error to force past
The web stack serves an old version	<code>docker compose up -d</code> reused a cached image	Always <code>--build</code> , then check <code>/version</code>
<code>OAK-RUNNER-REVOCATION</code> on every dispatch	The revocation manifest is missing or the notice set does not match it	Re-publish from the control plane (<code>oak revoke-approval</code> , or a fresh dispatch establishes an empty manifest). Do not hand-delete notices — the mismatch is the protection working
<code>OAK-RUNNER-REPLAY</code> on every dispatch	The consumed-nonce ledger in <code>\$OAK_RUNNER_HOME</code> is corrupt; it deliberately refuses rather than reading as empty	Inspect <code>consumed-nonces.json</code> ; remove it only if you accept that previously burned nonces become replayable
<code>OAK-WORKSPACE-CORRUPT</code> on every command	Manifest unreadable, or written by a format this build does not know	Restore from your last <code>oak export</code> — this is why the manual says export before upgrading
<code>run-once</code> exited 0 but printed <code>denied <dispatch>: <reason></code>	Denying a dispatch is the runner working correctly; exit status reports whether the runner ran, not whether anything succeeded	Read the signed messages (<code>oak ingest</code>) or the journal, and look the denial code up in the chapter 5 table or <code>docs/error-codes.md</code>
Runner exits 64 or 70 with a code on stderr	The runner itself could not run: 64 is a missing required environment variable, 70 a configuration or verification refusal that stopped it before processing (e.g. <code>OAK-RUNNER-CONFIG</code>)	Fix the environment or target profile it names — every <code>OAK-*</code> code is documented with its raise site in <code>docs/error-codes.md</code>

The full runbook is `docs/operations.md` ; the complete generated error-code reference is `docs/error-codes.md` .

Uninstalling completely

```
# 1 · Stack, volumes (your data), OAK-built images, fixture containers
docker compose down --volumes --remove-orphans
docker image rm -f $(docker image ls -q 'oak-community-*') 2>/dev/null || true
docker rm -f $(docker ps -aq --filter "label=oak.fixture=true") 2>/dev/null || true

# 2 · Home-directory state: PRIVATE KEYS, runner journals, consumed nonces
rm -rf ~/.oak

# 3 · Workspaces you created (you chose these paths; OAK does not track them),
#     and chapter 5's trust directory and mailbox, which hold DEVELOPMENT PRIVATE
#     KEYS
rm -rf ./workspace ./runner-demo ./case-export ./restored ./trust ./mailbox

# 4 · Build output, venv and caches inside the checkout
make clean-all

# 5 · Confirm — run this BEFORE step 6, which deletes the script itself
python scripts/check_clean_machine.py

# 6 · Finally the checkout, if you no longer want the source
cd .. && rm -rf OAK
```

`~/.oak` holds private signing keys, the runner's journal, consumed nonces and its revocation-sequence mark — export anything you want to keep first. It also holds any model-provider key stored in the file backend, under `~/.oak/credentials`, plus the model selection in `~/.oak/models`. A key stored in the operating-system keychain is *not* under `~/.oak` and survives this: remove those first, one family at a time, with `oak models remove-key <family>`. Chapter 5 placed its trust directory and mailbox in the repository root and its runner home inside `runner-demo/`: `trust/` holds development private keys, which is why step 3 names it explicitly rather than leaving it to the checkout's removal. Shared caches (`~/.cache/uv`, the pnpm store, Playwright browsers) and the third-party images OAK pulled (`postgres:17.6-alpine`, `python:3.13.12-slim`, `node:24.18.0-alpine`, `nginxinc/nginx-unprivileged:1.29.1-alpine`) are shared Docker and toolchain state, yours to keep or remove with `docker image rm`. The full runbook version, including the named-volume and release-image variants, is `docs/operations.md § Uninstall`. Step 5 reports any OAK-related state that survived — run it while the checkout still exists, because step 6 removes the script along with everything else.

Colophon: how this manual is built

The source of this manual is `docs/manual/manual.html`; the PDF is a rendering of it. Both are committed. To rebuild:

```
# screenshots, from a live journey against the Compose stack
docker compose up -d --build
OAK_MANUAL_SCREENSHOTS=1 pnpm --dir web exec playwright test e2e/manual-screens.spec.ts

# the PDF, using the same pinned Chromium the e2e suite uses
pnpm --dir web exec node ../docs/manual/build_manual.mjs
```

Every command in chapters 3 and 5 was verified by running the full journeys against this exact tree, and the expected-output excerpts are what those runs printed. The repository's end-to-end suites (`tests/e2e/test_cli.py`, `tests/e2e/test_runner_journey.py`) exercise the same invocations continuously — except `oak revoke-approval`, which is covered by the revocation integration suite (`tests/integration/test_runner_revocation.py`) instead. The diagrams are inline SVG in the source. This manual describes OAK Community 0.7.1 — a local-first developer release with no production or customer readiness claim; where this document and `docs/security/residual-risk.md` could ever disagree, the register wins.